Example1 - Databricks Spark Sequel:

These queries are used in Power BI Validation to compare what is in the pbi semantic model vs what is in Databricks gold medallion layer.

```sql
--Joins, Conditions, Aggregate
SELECT p.period_description,
sum(dollar_sales),
sum(dollar_sales_ya),
sum(unit_sales),
sum(unit_sales_ya),
sum(rom_dollar_sales),
sum(rom_dollar_sales_ya),
sum(rom_unit_sales),
sum(rom_unit_sales_ya),
sum(mulo_rom_dollar_sales),
sum(mulo_rom_dollar_sales_ya),
sum(mulo_rom_unit_sales),
sum(mulo_rom_unit_sales_ya)
from dnap_cpg_sales_nghq_prod.gold_nghq_circana.reporting_view_fact_sales_aggregate as f
JOIN dim_period as p ON p.period_key = f.period_key
JOIN dim_product as i ON i.product_key = f.product_key
JOIN dim_geography as g ON g.geography_key = f.geography_key
group by p.period_description
```

```sql
--Count Categories & Super Categories
--Joins, Conditions, Aggregate
SELECT
COUNT(Distinct super_category),
COUNT(Distinct category)
FROM dnap_cpg_sales_nghq_prod.gold_nghq_nielsen.reporting_view_fact_sales_aggregate AS f
JOIN dim_period AS p ON p.period_key = f.period_key
JOIN dim_product as i ON i.product_key = f.product_key
JOIN dim_geography as g ON g.geography_key = f.geography_key
WHERE g.panel_pos = 'POS'
AND p.period_type IS NOT NULL AND p.period_type <> 'Weekly' AND p.period_type <> 'Unknown'
AND g.rso <> 'Albertsons'
AND g.class <> 'Total US Drug'
AND g.geography NOT LIKE 'Rem%'
```

Example2 - Databricks Spark Sequel:

```sql
--Count Fact Rows by Period (Doesn't need to be exact but does need to be close!)
SELECT
 CASE
  WHEN p.period_description LIKE 'Latest 4%'  THEN CONCAT('1 ', p.period_description)
  WHEN p.period_description LIKE 'Latest 13%' THEN CONCAT('2 ', p.period_description)
  WHEN p.period_description LIKE 'Latest 26%' THEN CONCAT('3 ', p.period_description)
  WHEN p.period_description LIKE 'Latest 52%' THEN CONCAT('4 ', p.period_description)
  WHEN p.period_description LIKE '%YTD %' THEN CONCAT('5 ', p.period_description)
  ELSE p.period_description
 END AS period_short,
 COUNT(*) AS total_row_count
from dnap_cpg_sales_nghq_prod.gold_nghq_nielsen.reporting_view_fact_sales_aggregate as f
JOIN dim_period AS p ON p.period_key = f.period_key
JOIN dim_product AS i ON i.product_key = f.product_key
JOIN dim_geography AS g ON g.geography_key = f.geography_key
WHERE g.panel_pos = 'POS'
AND p.period_type IS NOT NULL AND p.period_type <> 'Weekly' AND p.period_type <> 'Unknown'
AND g.rso <> 'Albertsons'
AND g.class <> 'Total US Drug'
AND g.geography NOT LIKE 'Rem%'
GROUP BY
 CASE
  WHEN p.period_description LIKE 'Latest 4%'  THEN CONCAT('1 ', p.period_description)
  WHEN p.period_description LIKE 'Latest 13%' THEN CONCAT('2 ', p.period_description)
  WHEN p.period_description LIKE 'Latest 26%' THEN CONCAT('3 ', p.period_description)
  WHEN p.period_description LIKE 'Latest 52%' THEN CONCAT('4 ', p.period_description)
  WHEN p.period_description LIKE '%YTD %' THEN CONCAT('5 ', p.period_description)
  ELSE p.period_description
 END
ORDER BY period_short ASC;
```

```sql
--Totals by Period
SELECT
 CASE
  WHEN p.period_description LIKE 'Latest 4%'  THEN CONCAT('1 ', p.period_description)
  WHEN p.period_description LIKE 'Latest 13%' THEN CONCAT('2 ', p.period_description)
  WHEN p.period_description LIKE 'Latest 26%' THEN CONCAT('3 ', p.period_description)
  WHEN p.period_description LIKE 'Latest 52%' THEN CONCAT('4 ', p.period_description)
  WHEN p.period_description LIKE '%YTD %' THEN CONCAT('5 ', p.period_description)
  ELSE p.period_description
 END AS period_short,
ROUND(SUM(dollar_sales_in_retail_measurement_channel),0) as `$ - RM Channel`,
ROUND(SUM(dollar_sales_in_retail_measurement_channel_ya),0) as `$ YA - RM Channel`,
SUM(units_sold_in_retail_measurement_channel) as `Units - RM Channel`,
SUM(units_sold_in_retail_measurement_channel_ya) as `Units YA - RM Channel`,
ROUND(SUM(dollar_sales_in_retail_measurement_xaoc),0) as `$ - RM xAOC`,
ROUND(SUM(dollar_sales_in_retail_measurement_xaoc_ya),0) as `$ YA - RM xAOC`,
SUM(units_sold_in_retail_measurement_xaoc) as `Units - RM xAOC`,
SUM(units_sold_in_retail_measurement_xaoc_ya) as `Units YA - RM xAOC`
from dnap_cpg_sales_nghq_prod.gold_nghq_nielsen.reporting_view_fact_sales_aggregate as f
JOIN dim_period AS p ON p.period_key = f.period_key
JOIN dim_product AS i ON i.product_key = f.product_key
JOIN dim_geography AS g ON g.geography_key = f.geography_key
WHERE g.panel_pos = 'POS'
AND p.period_type IS NOT NULL AND p.period_type <> 'Weekly' AND p.period_type <> 'Unknown'
AND g.rso <> 'Albertsons'
AND g.class <> 'Total US Drug'
AND g.geography NOT LIKE 'Rem%'
GROUP BY ALL
ORDER BY period_short ASC;
```

Example3 - Databricks Spark Sequel:

```
-------Total Weekly Row Count by week, last 20 Weeks-------Gold Layer
SELECT
p.period_description,
COUNT(*) AS total_row_count
FROM dnap_cpg_sales_nghq_prod.gold_nghq_nielsen.reporting_view_fact_sales_weekly AS f
JOIN dim_period AS p ON p.period_key = f.period_key
WHERE p.week_number BETWEEN 1 AND 20
GROUP BY p.period_description
ORDER BY p.period_description ASC;
```

Example4 – Sequel Server Management Studio (SSMS)

```sql
--Simple Conditions
SELECT *
FROM [Bona].[Master].[dimItem]
WHERE Country = 'US'
AND (Surface IN ('HWF','STL') OR Offer = 'BONUS')
AND ItemKey >= 140927
ORDER BY ItemKey ASC

--Alias
SELECT
MIN(UnitSales) AS 'UnitSales',
Category
FROM [Bona].[Master].[factData] as f
JOIN [Bona].Master.dimItem as i ON i.ItemKey =
f.ItemKey
GROUP BY Category

--Joins
SELECT
ds.DataSource,
i.Category,
SUM(UnitSales) AS 'Unit Sales'
FROM [Bona].[Master].[factData] as f
JOIN [Bona].Master.dimItem as i ON i.ItemKey =
f.ItemKey
JOIN Bona.Master.dimDataSource as ds ON
ds.DataSourceKey = f.DataSourceKey
JOIN Bona.Master.dimDate as d ON d.DateKey = f.DateKey
WHERE d.Date BETWEEN '2020-01-01' AND '2022-01-01'
GROUP BY ds.DataSource, i.Category
ORDER BY ds.DataSource ASC, [Unit Sales] DESC
```

```sql
--Simple Calculation
SELECT
ds.DataSource,
i.ItemDescription,
SUM(f.UnitSales) AS 'Units',
AVG(i.UnitCost) AS 'Average_Unit_Cost',
SUM(f.UnitSales) * AVG(i.UnitCost) AS 'Total Cost'
FROM [Bona].[Master].[factData] as f
JOIN [Bona].Master.dimItem as i ON i.ItemKey = f.ItemKey
JOIN Bona.Master.dimDataSource as ds ON ds.DataSourceKey = f.DataSourceKey
JOIN Bona.Master.dimDate as d ON d.DateKey = f.DateKey
WHERE d.Date BETWEEN '2020-01-01' AND '2022-01-01'
GROUP BY ds.DataSource, i.ItemDescription
ORDER BY ds.DataSource ASC


--DIVISION with Div/0 handling using “CASE WHEN”
SELECT
ds.DataSource,
i.ItemDescription,
SUM(RetailDollars) AS 'Dollars',
SUM(f.UnitSales) AS 'Units',
CASE
    WHEN (SUM(RetailDollars) IS NULL OR SUM(f.UnitSales) IS NULL)
OR (SUM(RetailDollars) = 0 OR SUM(f.UnitSales) = 0)
THEN 0
    ELSE SUM(RetailDollars)/SUM(f.UnitSales)
  END AS Avg_Price

FROM [Bona].[Master].[factData] as f
JOIN [Bona].Master.dimItem as i ON i.ItemKey = f.ItemKey
JOIN Bona.Master.dimDataSource as ds ON ds.DataSourceKey = f.DataSourceKey
JOIN Bona.Master.dimDate as d ON d.DateKey = f.DateKey
WHERE d.Date BETWEEN '2020-01-01' AND '2022-01-01'
GROUP BY ds.DataSource, i.ItemDescription
ORDER BY ds.DataSource ASC, Avg_Price DESC
```

Example5 – Sequel Server Management Studio (SSMS)

```
--Joins, Conditions, Groups
SELECT
d.Date,
BilledQuantity
FROM [ADV_Encompass].[KeurigDrPepper].[factDataAWG] f
INNER JOIN [ADV_Encompass].[Master].[dimDate] d ON d.DateKey = f.DateKey
INNER JOIN [ADV_Encompass].[Master].[dimStore] s ON s.StoreKey = f.StoreKey
INNER JOIN [ADV_Encompass].[KeurigDrPepper].[dimItem_AWG] i ON i.ItemKey = f.ItemKey
WHERE d.Date BETWEEN '03/01/2024' AND '05/10/2024'
AND ItemCode = '0309641'
AND EquityGroupCode IN ('1870','7538','7401')
AND s.RptLvlFilter = 'INCLUDE'
GROUP BY d.Date, BilledQuantity
ORDER BY d.Date ASC
```